UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

# LABORATORIO DE COMPUTACIÓN IV

Unidad 2:

Docker

Material de Estudio

2° Año – 4° Cuatrimestre

CC BY ND NC

# Índice

**Docker** 2

Virtualización Clásica ..... 2

Docker y Contenedores ..... 3

Diferencias entre Docker y Virtualización Clásica ..... 3

¿Utiliza Docker virtualización? ..... 5

**Arquitectura de Docker** 6

**Componentes de Docker** 7

Imágenes de Docker ..... 7

Cache en las imágenes de Docker ..... 8

Contenedor de Docker ..... 8

Volumen de Docker ..... 8

Docker Network ..... 9

**Beneficios de utilizar Docker** 10

**Docker Compose** 11

Estructura del Docker Compose ..... 12

**BIBLIOGRAFÍA** 14

## Docker

Habiendo finalizado el desarrollo de un servicio o aplicación es necesario poder desplegarlos, es decir, ejecutar y disponibilizar el software, para que pueda ser usado y accedido por los usuarios. Para ejecutar estos servicios existen diversas formas y estrategias para realizarlo, por ej: se puede correr localmente en una computadora y que los usuarios lo accedan vía red. Pero esto generaría dificultad para escalar los recursos, no podríamos asegurar que esté ejecutandose todo el tiempo, dificultaría poder controlar los distintos servicios corriendo en toda la organización, entre muchas más dificultades.

Por lo tanto, la solución necesaria es ejecutar nuestro servicio en un contexto controlado y dedicado únicamente para nuestro servicio y que podamos monitorizar su comportamiento. Para esto, antes se instalaban máquinas virtuales en los servidores y luego se corrían las aplicaciones en ellas.

Una máquina virtual es una tecnología que permite la creación y ejecución de entornos virtuales aislados dentro de un mismo sistema físico. En lugar de ejecutar una única instancia de un sistema operativo directamente en el hardware, la virtualización permite ejecutar múltiples instancias aisladas de sistemas operativos en una única máquina física.

### Virtualización Clásica

En la virtualización clásica, se utiliza un hipervisor, que es un software que crea y gestiona múltiples máquinas virtuales en una máquina física, haciéndose cargo de la gestión de sus recursos.

Cada VM tiene su propio sistema operativo, que se ejecuta independientemente del sistema operativo anfitrión. Esto requiere una cantidad significativa de recursos ya que cada VM debe incluir su propio kernel y sistema operativo completo. Debido a esto, la virtualización clásica puede ser pesada en términos de consumo de recursos y tiempo de inicio de las máquinas virtuales.

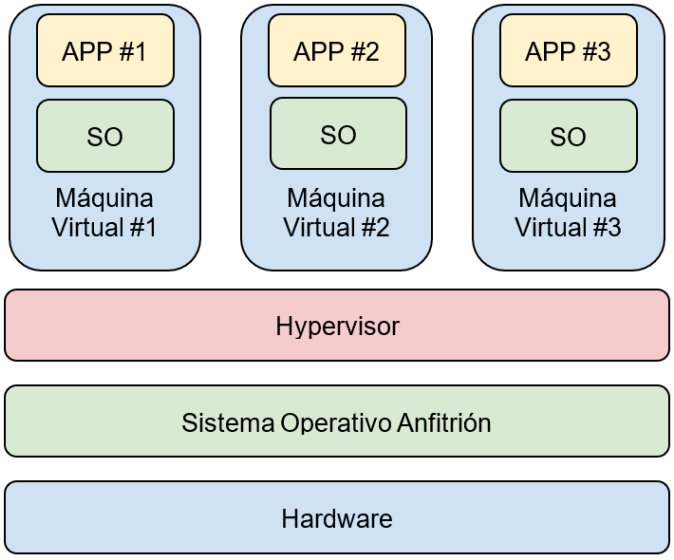


**Imagen 1**: Elaboración propia.

## Docker y Contenedores

Aquí es donde Docker entra en juego. Docker es una plataforma de contenedores que proporciona una forma eficiente de empacar, distribuir y ejecutar aplicaciones junto con todas sus dependencias en un entorno aislado.

A diferencia de la virtualización clásica, Docker no utiliza una máquina virtual completa para cada aplicación. En su lugar, utiliza contenedores, que son instancias aisladas que comparten el mismo kernel del sistema operativo anfitrión. Cada contenedor es un proceso del sistema operativo anfitrión, por lo que la gestión de los recursos es más eficiente y el SO no reserva recursos para cada contenedor como lo hace una máquina virtual. De esta manera los recursos van siendo asignados a medida que cada proceso va necesitándolos.

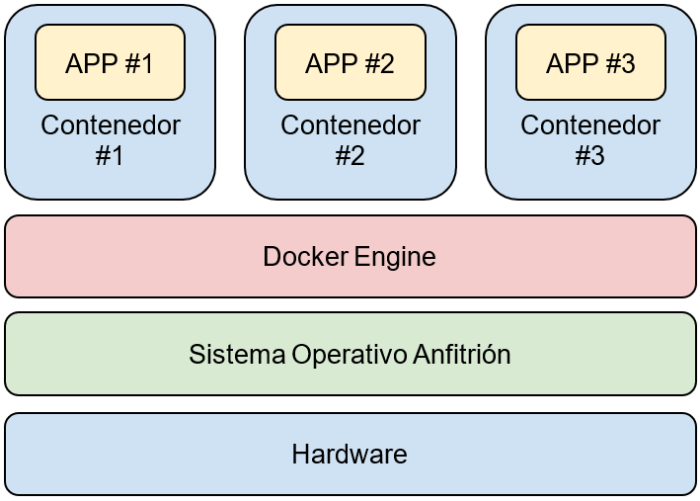


**Imagen 2**: Elaboración propia.

## Diferencias entre Docker y Virtualización Clásica

Las diferencias más grandes entre Docker y la virtualización clásica son las siguientes:

- **Eficiencia de recursos:**
  - *Docker*: Los contenedores Docker comparten el mismo sistema operativo subyacente y, por lo tanto, utilizan menos recursos en comparación con las máquinas virtuales. Además los recursos del SO se asignan a medida que cada contenedor los van necesitando.
  - *Virtualización clásica*: Las máquinas virtuales ejecutan sistemas operativos completos, lo que puede llevar a un uso más intensivo de recursos debido a la duplicación de sistemas operativos. Por la manera en que trabaja el Hypervisor los recursos que va a utilizar cada maquina virtual debe ser asignado previamente, independientemente de uso uso posterior.

- **Rendimiento:**
  - Docker: Los contenedores comparten el kernel del sistema operativo anfitrión, lo que resulta en un mejor rendimiento generalmente.
  - Virtualización clásica: Cada máquina virtual tiene su propio kernel, lo que puede llevar a cierta sobrecarga y una menor eficiencia en términos de rendimiento.
- **Tiempo de inicio:**
  - Docker: Los contenedores se inician mucho más rápido que las máquinas virtuales, lo que es beneficioso para la escalabilidad y la implementación rápida.
  - Virtualización clásica: Las máquinas virtuales pueden requerir más tiempo para iniciarse debido a la necesidad de cargar y configurar un sistema operativo completo.
- **Portabilidad:**
  - Docker: Los contenedores Docker encapsulan aplicaciones y dependencias, lo que facilita la portabilidad entre diferentes entornos.
  - Virtualización clásica: Las máquinas virtuales son más pesadas y pueden tener problemas de compatibilidad al moverse entre diferentes plataformas o entornos. Es más difícil llevar un control de las versiones de los distintos componentes que se ejecutan en ella, por ejemplo: la versión del SO, de las distintas dependencias de la aplicación que está corriendo, la versión de la aplicación, etc.
- **Aislamiento:**
  - Docker: Aunque los contenedores comparten el kernel, Docker utiliza tecnologías de aislamiento para proporcionar un alto grado de separación entre los contenedores, lo que limita la interferencia entre ellos.
  - Virtualización clásica: Las máquinas virtuales proporcionan un nivel más alto de aislamiento, ya que ejecutan sistemas operativos completamente separados.

Se puede resumir en la siguiente tabla:

| Característica | Docker | Virtualización Clásica |
|---|---|---|
| Eficiencia de recursos | Más livianos, comparten SO subyacente. | Sistema operativo completo independiente del anfitrión (más pesado) |
| Rendimiento | Mayor rendimiento, por la eficiencia en el uso de sus recursos. | Rendimiento menor |
| Tiempo de inicio | Inicio rápido | Inicio más lento |
| Portabilidad | Altamente portátil entre entornos | Menos portabilidad, más pesado |
| Aislamiento | Aislamiento usando tecnologías del SO | Aislamiento profundo con VMs |

**Tabla 1**: Elaboración propia.

### ¿Utiliza Docker virtualización?

**Docker no utiliza virtualización en el sentido tradicional de máquinas virtuales completas**. En cambio, utiliza tecnologías del kernel de Linux para lograr el aislamiento entre contenedores. Esto permite que múltiples contenedores compartan recursos mientras permanecen completamente aislados entre sí y del sistema operativo anfitrión.

Cada contenedor de Docker es esencialmente un proceso o conjunto de procesos que se ejecutan de manera aislada del resto del sistema, pero comparten recursos del sistema operativo subyacente, como el kernel.

Docker aborda los problemas de la virtualización tradicional al proporcionar un entorno ligero, eficiente y portátil para empaquetar y ejecutar aplicaciones junto con sus dependencias. Aunque no utiliza máquinas virtuales completas, emplea tecnologías de aislamiento del sistema operativo para lograr una separación efectiva entre los contenedores.

## Arquitectura de Docker

Docker consta de varios componentes interconectados que trabajan juntos para permitir la creación, el despliegue y la administración de contenedores. Estos componentes forman la base de cómo Docker opera y proporciona sus funcionalidades. Aquí tienes una descripción de los componentes clave y sus funciones en la arquitectura de Docker:

- **Docker Daemon (Dockerd)**: Es un proceso que se ejecuta en el sistema anfitrión y es responsable de administrar los contenedores. El daemon escucha las solicitudes de la API de Docker y maneja la creación, ejecución y gestión de los contenedores. Actúa como el núcleo del sistema Docker.
- **Docker Client (Docker CLI - Command Line Interface)**: Es una interfaz de línea de comandos o una API que permite a los usuarios interactuar con el daemon de Docker. Los usuarios emiten comandos a través del cliente para crear, ejecutar y gestionar contenedores, así como para gestionar imágenes y configuraciones.
- **Imágenes de Docker (Docker Images)**: Las imágenes son plantillas o snapshots que contienen una aplicación, sus dependencias y todas configuraciones necesarias para ejecutarse. Las imágenes son necesarias como base para crear contenedores. Se almacenan en el Registro de Docker, que puede ser el registro público de Docker Hub o un registro privado.
- **Contenedores de Docker (Docker Containers)**: Los contenedores son instancias en tiempo de ejecución de imágenes. Cada contenedor es un entorno aislado que comparte el kernel del sistema anfitrión. Contienen todos los componentes necesarios para ejecutar una aplicación, incluidos procesos, sistema de archivos y variables de entorno.
- **Registros de Docker (Docker Registries)**: Los registros son repositorios que almacenan imágenes de Docker. Docker Hub es un registro público ampliamente utilizado, pero también es posible configurar registros privados para almacenar imágenes personalizadas y sensibles. El docker daemon cuando necesita descargar una imagen la busca en el registry y la descarga.

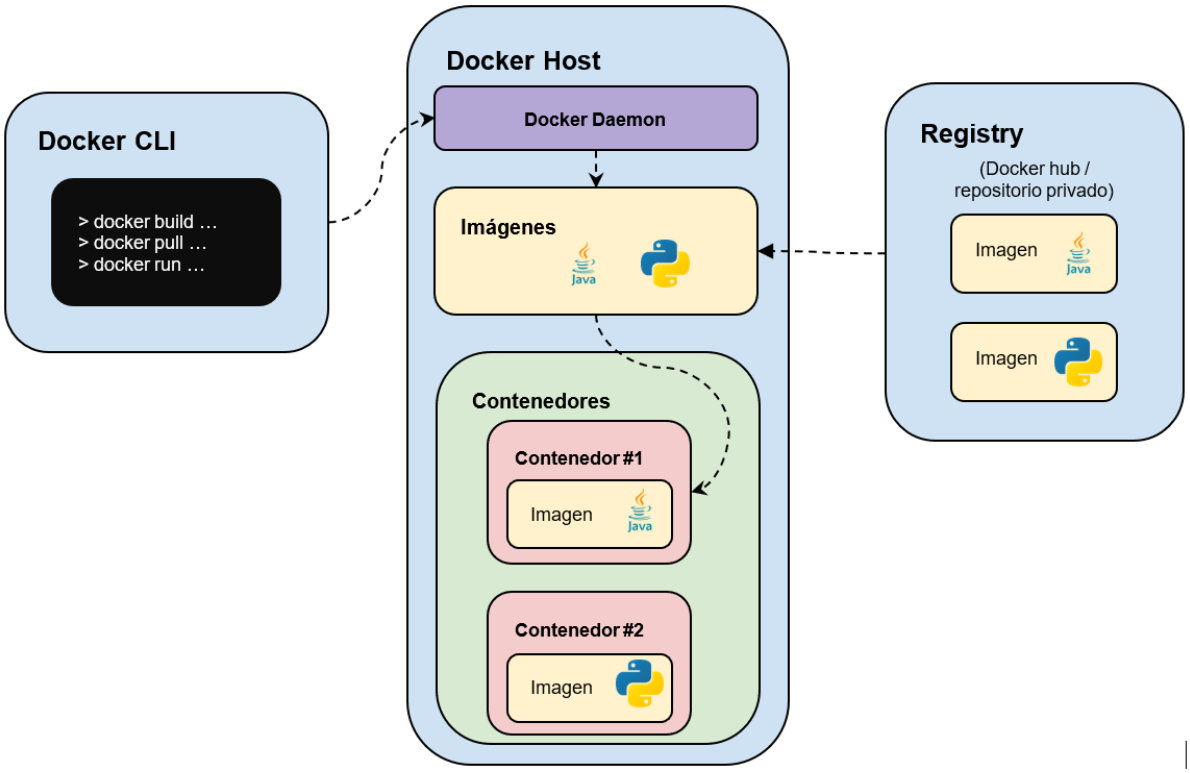


**Imagen 3**: Extraída de internet.

## Componentes de Docker

### Imágenes de Docker

Las imágenes de Docker son paquetes ligeros, independientes y ejecutables que contienen todo lo necesario para ejecutar una pieza de software, incluido el código, el entorno de ejecución, las bibliotecas, las variables de entorno y la configuración. Las imágenes de Docker son la base para crear y ejecutar contenedores.

Una imagen de Docker es esencialmente una instantánea estática (**snapshot**) y no modificable de un sistema de archivos que puede contener un sistema operativo completo o una aplicación específica junto con sus dependencias. Estas imágenes son creadas a partir de un archivo llamado "**Dockerfile**," que especifica todos los pasos necesarios para construir la imagen, como qué sistema operativo base usar, qué paquetes instalar y cómo configurar el entorno.

Las imágenes de Docker son altamente portátiles, lo que significa que una vez que se crea una imagen, se puede distribuir y ejecutar en cualquier entorno que admita Docker, ya sea en una máquina de desarrollo, un servidor de pruebas o un entorno de producción. Esto facilita la replicación y el despliegue consistente de aplicaciones en diferentes entornos.

### Cache en las imágenes de Docker

Las imágenes de Docker también siguen un modelo de capas. Cada instrucción en el Dockerfile crea una nueva capa en la imagen, lo que permite reutilizar partes comunes de las imágenes y mantener un control eficiente de versiones y cambios. Esto es especialmente útil para optimizar el uso del almacenamiento y para permitir actualizaciones y parches eficientes.

Estas capas pueden ser guardadas de manera independiente en un caché, permitiendo que sea reutilizadas por otras imágenes. Esto agiliza el proceso de construcción al evitar la necesidad de repetir tareas que ya se han realizado anteriormente, como la descarga de paquetes y la ejecución de comandos.

Cuando Docker construye una imagen, verifica si las capas ya existen en el caché local. Si una capa específica ya se ha construido previamente y sus condiciones no han cambiado, Docker reutiliza esa capa desde el caché en lugar de volver a construirla. Esto acelera significativamente el proceso de construcción.

El caché de imágenes es beneficioso en situaciones donde estás iterando en el desarrollo y realizando cambios en un Dockerfile. Si solo modificas una parte del Dockerfile y el resto permanece igual, Docker utilizará el caché para reutilizar las capas existentes y sólo construirá las partes que han cambiado. Esto ahorra tiempo y recursos en comparación con una construcción completa desde cero.

## Contenedor de Docker

Un contenedor de Docker es una instancia aislada y ejecutable de una imagen de Docker. En términos más simples, es un entorno ligero y portátil que contiene una aplicación y todas sus dependencias necesarias para funcionar de manera coherente en cualquier entorno compatible con Docker. Los contenedores permiten empaquetar, distribuir y ejecutar aplicaciones de manera eficiente y consistente.

## Volumen de Docker

Los volúmenes en Docker son una forma de persistir y compartir datos entre contenedores y el sistema anfitrión, independientemente del ciclo de vida del contenedor. En esencia, los volúmenes son puntos de montaje en el sistema de archivos del contenedor que apuntan a ubicaciones específicas en el sistema anfitrión o en otros contenedores. Esto permite que los datos se conserven incluso cuando un contenedor se detiene o se elimina.

Los volúmenes tienen varias ventajas clave:

- **Persistencia de Datos**: Los datos almacenados en un volumen persisten más allá del ciclo de vida del contenedor. Esto es especialmente útil para bases de datos, archivos de configuración y cualquier otro dato que necesite sobrevivir a la terminación del contenedor.
- **Compartición de Datos**: Los volúmenes permiten compartir datos entre varios contenedores. Varios contenedores pueden montar el mismo volumen, lo que facilita la colaboración y la comunicación entre contenedores.
- **Aislamiento de Aplicación**: Aunque los contenedores son efímeros y pueden ser destruidos y reemplazados fácilmente, los volúmenes permiten que los datos valiosos o críticos se mantengan fuera del ciclo de vida del contenedor.
- **Backup y Restauración**: Los volúmenes facilitan la realización de copias de seguridad y la restauración de datos, ya que los datos persisten en ubicaciones externas al contenedor.

### Docker Network

Docker Network (Red de Docker) es un componente de Docker que permite la comunicación entre contenedores en un entorno de contenerizado. Las Docker Networks facilitan la creación de redes virtuales aisladas en las que los contenedores pueden comunicarse entre sí y con otros servicios, ya sea dentro de la misma máquina o en diferentes máquinas en un entorno de clúster.

Las Docker Networks permiten:

- **Aislamiento**: Cada red de Docker proporciona un aislamiento lógico para los contenedores que se encuentran en ella. Esto significa que los contenedores en diferentes redes no pueden comunicarse directamente entre sí, lo que mejora la seguridad y el control.
- **Comunicación entre Contenedores**: Los contenedores que se encuentran en la misma red pueden comunicarse entre sí utilizando sus nombres de servicio como nombres de host. Esto simplifica la comunicación y reduce la necesidad de conocer direcciones IP específicas.
- **Conectividad Externa**: Puedes configurar una red de Docker para permitir la conectividad entre los contenedores y recursos externos, como sistemas en la red del host o en otras redes.

## Beneficios de utilizar Docker

El uso de Docker proporciona una serie de beneficios clave para el desarrollo, la implementación y la administración de aplicaciones. Aquí están algunos de los principales beneficios de utilizar Docker:

- **Aislamiento y Consistencia**: Docker encapsula aplicaciones y sus dependencias en contenedores, lo que garantiza un alto grado de aislamiento entre las aplicaciones y evita conflictos entre ellas. Esto también asegura la consistencia en diferentes entornos, eliminando problemas de "funciona en mi máquina".
- **Portabilidad**: Las imágenes de Docker contienen todo lo necesario para ejecutar una aplicación, incluidas bibliotecas y configuraciones. Esto hace que las aplicaciones sean altamente portátiles y puedan ejecutarse en cualquier entorno que admita Docker, desde tu máquina local hasta la nube.
- **Despliegue Rápido**: Los contenedores de Docker se inician en cuestión de segundos, lo que acelera el proceso de despliegue y permite una escalabilidad rápida para manejar cambios en la carga de trabajo.
- **Gestión Simplificada**: Docker simplifica la administración de aplicaciones y servicios. Puedes gestionar múltiples contenedores utilizando herramientas como Docker Compose o Kubernetes, lo que facilita la orquestación, la escalabilidad y la actualización de aplicaciones.
- **Eficiencia de Recursos**: Los contenedores comparten el mismo kernel del sistema operativo anfitrión, lo que reduce la sobrecarga en comparación con las máquinas virtuales completas. Esto resulta en un uso más eficiente de los recursos y permite ejecutar más aplicaciones en la misma infraestructura.
- **Escalabilidad**: Docker facilita la escalabilidad horizontal al permitir la replicación de contenedores según la demanda. Esto es esencial para manejar cambios en el tráfico y mantener el rendimiento de las aplicaciones.
- **Versionado y Rollbacks**: Las imágenes de Docker se pueden versionar y etiquetar, lo que facilita el seguimiento de cambios y permite realizar rollbacks en caso de problemas, es decir, volver a una versión anterior a la que se está intentando desplegar. Esto es especialmente útil en situaciones de implementación y pruebas.

- **Colaboración Mejorada**: Las imágenes de Docker contienen todas las dependencias necesarias para ejecutar una aplicación, lo que simplifica la colaboración entre desarrolladores y equipos de operaciones. Las discrepancias en los entornos de desarrollo se reducen significativamente.
- **Entornos de Desarrollo Reproducibles**: Docker asegura que los entornos de desarrollo sean consistentes entre diferentes desarrolladores y entornos. Esto elimina problemas causados por diferencias en las configuraciones locales.
- **Seguridad y Aislamiento**: Los contenedores de Docker ofrecen un nivel adicional de seguridad al aislar aplicaciones y procesos. Esto ayuda a prevenir que vulnerabilidades en una aplicación afecten a otras.

## Docker Compose

Docker Compose es una herramienta que permite definir y ejecutar aplicaciones multi-contenedor de manera más sencilla. En lugar de manejar y orquestar varios contenedores individualmente, Docker Compose te permite definir la configuración de tus contenedores en un archivo de texto con formato YAML (https://docs.fileformat.com/es/programming/yaml/) y luego lanzar y administrar toda la aplicación con un solo comando. Esto es particularmente útil cuando tu aplicación consta de varios servicios que necesitan comunicarse entre sí.

Las principales características y beneficios de Docker Compose son:

- **Definición de Servicios**: En un archivo YAML llamado docker-compose.yml, puedes definir los servicios, contenedores y configuraciones necesarias para tu aplicación. Cada servicio puede consistir en uno o más contenedores.
- **Orquestación Sencilla**: Docker Compose permite definir relaciones y dependencias entre servicios. Por ejemplo, si tu aplicación web depende de una base de datos, puedes especificar que el servicio de la aplicación web se inicie después del servicio de la base de datos.
- **Entorno Reproducible**: Al definir toda la configuración en un archivo, Docker Compose asegura que todos los miembros del equipo tengan un entorno de desarrollo y pruebas consistente y reproducible.

- **Fácil Implementación y Escalabilidad**: Puedes utilizar un solo comando para construir, iniciar y administrar todos los contenedores definidos en el archivo docker-compose.yml. Esto facilita la implementación y escalabilidad de la aplicación.
- **Variables de Entorno**: Docker Compose admite el uso de variables de entorno en el archivo docker-compose.yml, lo que permite la personalización de configuraciones para diferentes entornos.
- **Comunicación entre Contenedores**: Docker Compose facilita la creación de redes internas para que los contenedores puedan comunicarse entre sí utilizando nombres de servicio en lugar de direcciones IP.
- **Gestión de Volúmenes**: Puedes definir volúmenes en Docker Compose para almacenar datos persistentes fuera de los contenedores, lo que evita la pérdida de datos cuando los contenedores se reinician o reemplazan.
- **Configuración Centralizada**: En lugar de tener que ejecutar múltiples comandos de Docker para cada servicio, Docker Compose centraliza todo en un solo archivo.

### Estructura del Docker Compose

El archivo de Docker Compose, usualmente llamado docker-compose.yml, se estructura utilizando una sintaxis YAML. Las principales partes de Docker Compose son:

1. **Version**: Define la versión de la sintaxis del archivo docker-compose.yml que estás utilizando. Esto afecta cómo se interpretan las instrucciones y opciones en el archivo.
2. **Services**: Esta sección es donde defines cada uno de los servicios que formarán parte de tu aplicación. Cada servicio es un contenedor o grupo de contenedores relacionados que trabajan juntos para proporcionar una funcionalidad específica.
    a) **Image**: Especifica la imagen Docker que se utilizará para crear el contenedor del servicio.
    b) **Container Name**: Opcionalmente, puedes asignar un nombre específico al contenedor.
    c) **Ports**: Define los puertos que deben exponerse desde el contenedor.

d) **Environment Variables**: Permite definir variables de entorno específicas para el contenedor.

e) **Volumes**: Indica volúmenes que deben montarse en el contenedor.

f) **Depends On**: Define las dependencias entre servicios, lo que asegura que un servicio se inicie después de que sus dependencias estén en funcionamiento.

3. **Networks**: Puedes definir redes personalizadas para tus servicios. Esto permite que los contenedores se comuniquen entre sí utilizando los nombres de servicio en lugar de direcciones IP.
4. **Volumes**: Aquí puedes definir volúmenes personalizados para tus servicios. Los volúmenes se pueden montar en los contenedores para persistir los datos más allá del ciclo de vida del contenedor.
5. **Environment Variables**: Puedes definir variables de entorno globales que serán aplicadas a todos los servicios en el archivo docker-compose.yml.

## BIBLIOGRAFÍA


- Página oficial de DOCKER (https://docs.docker.com/).


**Atribución-No Comercial-Sin Derivadas**

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.